

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

VICTOR HUGO DOS SANTOS CAVALCANTE

**SISTEMA DE GESTÃO DE UMA CAFETERIA: ESTUDO DE
CASO UTILIZANDO MONGODB**

CAMPOS DO JORDÃO

2026

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE
SÃO PAULO**

VICTOR HUGO DOS SANTOS CAVALCANTE

**SISTEMA DE GESTÃO DE UMA CAFETERIA: ESTUDO DE
CASO UTILIZANDO MONGODB**

Projeto NoSQL da matéria Banco de Dados II
em Análise e Desenvolvimento de Sistemas
pelo Instituto Federal de São Paulo - IFSP.

Orientador: Prof. Paulo Giovani de Faria
Zeferino

CAMPOS DO JORDÃO

2026

RESUMO

Este documento apresenta o desenvolvimento e a migração de um projeto de base de dados para o paradigma Não Relacional (NoSQL), focado na gestão sistémica de uma cafeteria fictícia denominada "Café Aconchego". O objetivo principal consiste em reestruturar um modelo relacional previamente desenvolvido para um modelo orientado a documentos, visando controlar operações essenciais como o catálogo de produtos, o registo de clientes e funcionários, o estado das mesas e as transações de vendas. A metodologia adotada fundamenta-se na utilização do Sistema de Gestão de Bases de Dados (SGBD) MongoDB, que emprega o formato BSON (JSON binário) para armazenamento. A modelagem proposta explora conceitos intrínsecos ao ecossistema NoSQL, como a desnormalização e a incorporação de dados (Embedded Documents), fundindo entidades relacionais distintas (como Vendas e Itens da Venda) em coleções únicas de alta performance. Os resultados obtidos demonstram a flexibilidade de esquema e a escalabilidade da solução através da execução de 30 consultas complexas na linguagem nativa do SGBD. Conclui-se que o MongoDB fornece uma infraestrutura robusta e perfeitamente alinhada com as necessidades de desenvolvimento de aplicações modernas no setor de retalho e serviços.

Palavras-Chave: Base de Dados Não Relacional; NoSQL; MongoDB; Modelagem Orientada a Documentos; Cafeteria.

ABSTRACT

This document presents the development and migration of a database project to the Non-Relational (NoSQL) paradigm, focusing on the system management of a fictional coffee shop called "Café Aconchego". The main objective is to restructure a previously developed relational model into a document-oriented model, aiming to manage essential operations such as the product catalog, customer and employee records, table status management, and sales transactions. The adopted methodology is based on the use of the MongoDB Database Management System (DBMS), which employs the BSON (Binary JSON) format for data storage. The proposed data model explores concepts inherent to the NoSQL ecosystem, such as denormalization and embedded documents, combining distinct relational entities (such as Sales and Sale Items) into unified high-performance collections. The results obtained demonstrate the schema flexibility and scalability of the solution through the execution of 30 complex queries using the database system's native query language. It is concluded that MongoDB provides a robust infrastructure that is fully aligned with the development requirements of modern applications in the retail and service sectors.

Keywords: Non-Relational Database; NoSQL; MongoDB; Document-Oriented Modeling; Coffee Shop.

LISTA DE ILUSTRAÇÕES

FIGURA 1 – Consulta 1 (MongoDB)	17
FIGURA 2 – Consulta 2 (MongoDB)	18
FIGURA 3 – Consulta 3 (MongoDB)	19
FIGURA 4 – Consulta 4 (MongoDB)	20
FIGURA 5 – Consulta 5 (MongoDB)	20
FIGURA 6 – Consulta 6 (MongoDB)	21
FIGURA 7 – Consulta 7 (MongoDB)	21
FIGURA 8 – Consulta 8 (MongoDB)	22
FIGURA 9 – Consulta 9 (MongoDB)	23
FIGURA 10 – Consulta 10 (MongoDB)	24
FIGURA 11 – Consulta 11 (MongoDB)	24
FIGURA 12 – Consulta 12 (MongoDB)	25
FIGURA 13 – Consulta 13 (MongoDB)	26
FIGURA 14 – Consulta 14 (MongoDB)	27
FIGURA 15 – Consulta 15 (MongoDB)	27
FIGURA 16 – Consulta 16 (MongoDB)	28
FIGURA 17 – Consulta 17 (MongoDB)	29
FIGURA 18 – Consulta 18 (MongoDB)	29
FIGURA 19 – Consulta 19 (MongoDB)	30
FIGURA 20 – Consulta 20 (MongoDB)	30
FIGURA 21 – Consulta 21 (MongoDB)	31
FIGURA 22 – Consulta 22 (MongoDB)	31
FIGURA 23 – Consulta 23 (MongoDB)	32

FIGURA 24 – Consulta 24 (MongoDB)	33
FIGURA 25 – Consulta 25 (MongoDB)	34
FIGURA 26 – Consulta 26 (MongoDB)	34
FIGURA 27 – Consulta 27 (MongoDB)	35
FIGURA 28 – Consulta 28 (MongoDB)	36
FIGURA 29 – Consulta 29 (MongoDB)	37
FIGURA 30 – Consulta 30 (MongoDB)	38

LISTA DE ALGORITMOS

ALGORITMO 1 – Inserção de Dados nas Coleções (MongoDB)	15
ALGORITMO 2 – Consulta 1 (MongoDB)	17
ALGORITMO 3 – Consulta 2 (MongoDB)	18
ALGORITMO 4 – Consulta 3 (MongoDB)	19
ALGORITMO 5 – Consulta 4 (MongoDB)	19
ALGORITMO 6 – Consulta 5 (MongoDB)	20
ALGORITMO 7 – Consulta 6 (MongoDB)	21
ALGORITMO 8 – Consulta 7 (MongoDB)	21
ALGORITMO 9 – Consulta 8 (MongoDB)	22
ALGORITMO 10 – Consulta 9 (MongoDB)	22
ALGORITMO 11 – Consulta 10 (MongoDB)	23
ALGORITMO 12 – Consulta 11 (MongoDB)	24
ALGORITMO 13 – Consulta 12 (MongoDB)	25
ALGORITMO 14 – Consulta 13 (MongoDB)	26
ALGORITMO 15 – Consulta 14 (MongoDB)	26
ALGORITMO 16 – Consulta 15 (MongoDB)	27
ALGORITMO 17 – Consulta 16 (MongoDB)	28
ALGORITMO 18 – Consulta 17 (MongoDB)	28

ALGORITMO 19 – Consulta 18 (MongoDB)	29
ALGORITMO 20 – Consulta 19 (MongoDB)	30
ALGORITMO 21 – Consulta 20 (MongoDB)	30
ALGORITMO 22 – Consulta 21 (MongoDB)	31
ALGORITMO 23 – Consulta 22 (MongoDB)	31
ALGORITMO 24 – Consulta 23 (MongoDB)	32
ALGORITMO 25 – Consulta 24 (MongoDB)	33
ALGORITMO 26 – Consulta 25 (MongoDB)	33
ALGORITMO 27 – Consulta 26 (MongoDB)	34
ALGORITMO 28 – Consulta 27 (MongoDB)	35
ALGORITMO 29 – Consulta 28 (MongoDB)	35
ALGORITMO 30 – Consulta 29 (MongoDB)	36
ALGORITMO 31 – Consulta 30 (MongoDB)	37

SUMÁRIO

1	INTRODUÇÃO	09
1.1	Objetivos	09
1.2	Justificativa	10
1.3	Aspectos Metodológicos	10
1.4	Aporte Teórico	11
2	METODOLOGIA	12
2.1	Considerações Iniciais sobre o Projeto	12
2.2	Descrição do Modelo NoSQL Adotado	12
2.3	O SGBD Utilizado: MongoDB	13
2.4	O Modelo de Dados do MongoDB e a Estratégia de Modelagem	13
3	RESULTADOS OBTIDOS	15
3.1	Consultas Desenvolvidas	17
4	CONCLUSÃO	39

1 INTRODUÇÃO

Neste capítulo serão introduzidos todos os assuntos abordados por este documento. O crescente avanço da tecnologia e o volume massivo de informações geradas diariamente impulsionaram o surgimento de novas abordagens para o armazenamento de dados corporativos. Este trabalho aborda o cenário prático da "Café Aconchego", uma cafeteria de pequeno a médio porte cujas operações diárias englobam o controle de um catálogo de produtos, a gestão de estoque, o cadastro de clientes e funcionários, o controle de mesas no salão e o registro detalhado de comandas (vendas).

Anteriormente, o cenário da "Café Aconchego" era gerenciado por um banco de dados estritamente relacional (SQL). No entanto, a dinâmica ágil do setor alimentício frequentemente exige flexibilidade estrutural como a adição de observações únicas em pedidos, variações de coberturas ou novos métodos de pagamento, o que torna a rigidez das tabelas relacionais um obstáculo. Diante desse cenário, este projeto realiza a migração do sistema da cafeteria para o paradigma de banco de dados Não Relacional (NoSQL), visando explorar as vantagens da modelagem orientada a documentos para agregar valor e escalabilidade ao negócio.

1.1 Objetivos

O objetivo geral deste trabalho é projetar, implementar e validar um banco de dados não relacional (NoSQL) para gerenciar de forma integrada as operações da cafeteria "Café Aconchego", realizando a migração do seu modelo relacional pré-existente.

Para a consecução deste objetivo foram estabelecidos os objetivos específicos:

- Investigar e descrever o ecossistema dos bancos de dados NoSQL e suas diferentes tipologias de armazenamento;
- Remodelar o esquema de dados tradicional (tabelas e chaves estrangeiras) para o paradigma orientado a documentos;
- Implementar a estrutura física do banco utilizando o SGBD MongoDB;

- Executar e documentar 30 consultas na sintaxe nativa do MongoDB, extraindo informações gerenciais e operacionais relevantes para a tomada de decisão no cenário abordado.

1.2 Justificativa

A relevância deste estudo reside na necessidade constante de adequar as ferramentas tecnológicas às exigências de desempenho e flexibilidade das aplicações modernas. Os bancos de dados relacionais, embora consolidados, apresentam limitações de escalabilidade horizontal e rigidez de esquema. Num cenário comercial de *food service* moderno, onde a estrutura de um pedido pode variar frequentemente para um mesmo produto, a adoção de um modelo orientado a documentos proporciona uma enorme vantagem competitiva e operacional. Este projeto justifica-se, assim, pela aplicação prática dessas tecnologias emergentes em um estudo de caso realista, resolvendo gargalos de modelagem tradicionais e preparando o aluno para os desafios arquiteturais do mercado de trabalho contemporâneo.

1.3 Aspectos Metodológicos

Respondendo a *como* a pesquisa será conduzida, a metodologia de desenvolvimento baseia-se em uma abordagem de migração de paradigma de armazenamento e estudo de caso prático. Inicialmente, realizou-se uma pesquisa bibliográfica e documental para fundamentar os conceitos de NoSQL e do SGBD escolhido. Posteriormente, a etapa prática consistiu na análise crítica das regras de negócio originais do "Café Aconchego" e na aplicação da técnica de Desnormalização de Dados. Entidades fortemente acopladas no modelo relacional (como *Vendas* e *Itens da Venda*) foram traduzidas para coleções independentes e documentos aninhados (*Embedded Documents*) no MongoDB. Por fim, a validação

do método ocorreu através do terminal interativo (Mongosh), onde os scripts de população do banco e extração de dados foram rigorosamente testados.

1.4 Aporte Teórico

Respondendo a *quais ideias* o trabalho se baseia, a fundamentação teórica sustenta-se nos princípios dos bancos de dados NoSQL (*Not Only SQL*). Trata-se de uma classe de Sistemas de Gerenciamento de Bancos de Dados (SGBDs) desenhados para contornar as limitações do modelo relacional (estruturado sob a teoria de Edgar F. Codd), focando em alta disponibilidade, escalabilidade distribuída e esquemas de dados dinâmicos (*schema-less*). A arquitetura NoSQL divide-se primariamente em quatro grandes famílias de modelos de dados:

- **Chave-Valor:** O modelo mais elementar, estruturado como um dicionário. Cada item possui uma chave única que aponta para um valor (ex: Redis, DynamoDB).
- **Família de Colunas:** Organiza os dados em agrupamentos de colunas, sendo altamente otimizado para cenários com elevadas taxas de escrita e grandes volumes de dados distribuídos (ex: Apache Cassandra, HBase).
- **Grafos:** Especializado no armazenamento de relacionamentos complexos. Utiliza a Teoria dos Grafos com nós (entidades) e arestas (ligações) para representar e consultar redes altamente interligadas de forma performática (ex: Neo4j).
- **Orientado a Documentos:** Armazena e recupera dados na forma de documentos (frequentemente em formatos JSON, XML ou BSON). Este modelo baseia-se na ideia de agrupar todas as informações pertencentes a um determinado objeto de negócio em um único registro, o que elimina a necessidade de junções complexas (*joins*) durante a leitura de dados agregados (ex: MongoDB, CouchDB).

2 METODOLOGIA

Este capítulo detalha os procedimentos, as ferramentas e as decisões arquiteturais adotadas para a execução do projeto de migração estrutural. A metodologia segue uma abordagem de engenharia de software focada na transição do paradigma relacional para o não relacional, justificando as escolhas tecnológicas aplicadas ao estudo de caso da cafeteria "Café Aconchego".

2.1 Considerações Iniciais sobre o Projeto

O projeto partiu de uma base de dados relacional (SQL) previamente estruturada e normalizada, que gerenciava as entidades do estabelecimento: Clientes, Funcionários, Mesas, Produtos, Vendas e Itens da Venda. A consideração inicial fundamental para a migração foi a análise do comportamento de leitura e escrita do sistema (padrão de acesso aos dados). Notou-se que a operação mais crítica do sistema a consulta de uma comanda (venda) completa exigia o processamento de junções (*JOINS*) custosas entre as tabelas de Vendas, Itens da Venda e Produtos. Para otimizar o desempenho de leitura e garantir flexibilidade na adição de novas propriedades aos pedidos (como observações customizadas), definiu-se a necessidade de migrar a arquitetura para um modelo que suportasse a agregação de dados em uma única entidade lógica.

2.2 Descrição do Modelo NoSQL Adotado

Diante da necessidade do projeto, o modelo NoSQL selecionado foi o Orientado a Documentos. Este modelo difere drasticamente do padrão tabular ao armazenar os dados de forma encapsulada e hierárquica. Em vez de espalhar os atributos de uma entidade por várias tabelas e reconstruí-los através de chaves estrangeiras, o modelo de documentos agrupa todas as informações pertinentes a um objeto em um único registro autossuficiente. A grande vantagem acadêmica e comercial deste modelo é a diminuição da "fricção" (*Impedance Mismatch*) entre a aplicação e o banco de dados. Como as linguagens de programação modernas lidam com objetos de forma nativa, salvar esses objetos diretamente como

documentos elimina camadas complexas de tradução de dados, resultando em um desenvolvimento mais ágil e em uma manutenção simplificada.

2.3 O SGBD Utilizado: MongoDB

Para a implementação prática, o Sistema de Gerenciamento de Banco de Dados escolhido foi o MongoDB. Trata-se de um banco de dados NoSQL de código aberto, líder global na categoria de orientação a documentos. O MongoDB foi escolhido por oferecer um ecossistema maduro, alta disponibilidade e uma linguagem de consulta (*Query Language*) extremamente rica e expressiva. Além disso, a sua arquitetura *schema-less* (esquema dinâmico) permite que a cafeteria insira novos produtos com características inéditas (por exemplo, um produto com atributos de "tamanho" e "tipo de grão") sem a necessidade de alterar a estrutura do banco de dados inteiro ou adicionar colunas com valores nulos para os demais itens.

2.4 O Modelo de Dados do MongoDB e a Estratégia de Modelagem

Ao contrário dos bancos relacionais tradicionais, o modelo de dados do MongoDB utiliza o formato BSON (Binary JSON). O BSON é uma representação binária seriada de documentos JSON (JavaScript Object Notation). A escolha do BSON permite que o MongoDB suporte tipos de dados nativos avançados que não existem no JSON puro, como tipos de data (Date), dados espaciais e identificadores únicos nativos (ObjectId).

No MongoDB, a terminologia relacional sofre uma adaptação direta:

- **Banco de Dados** permanece Banco de Dados.
- **Tabelas** tornam-se Coleções (Collections).
- **Linhas/Registros** tornam-se Documentos (Documents).
- **Colunas** tornam-se Campos (Fields).

Estratégia de Desnormalização (O Novo Modelo Prático): A etapa mais crítica da metodologia foi o redesenho do esquema. Adotou-se a técnica de Documentos Aninhados (Embedded Documents), que consiste em armazenar dados relacionados

dentro de um único documento BSON por meio de Arrays (vetores) ou subdocumentos.

O banco de dados "cafe_aconchego" foi modelado com as seguintes coleções, absorvendo a carga do modelo relacional:

1. **funcionarios:** Coleção independente contendo o documento de cada colaborador, mantendo seus dados de admissão e cargo.
2. **mesas:** Coleção para registrar o mapeamento físico do salão, com capacidade e status de ocupação.
3. **produtos:** Coleção que compõe o catálogo, contendo nome, preço, categoria e estoque.
4. **clientes:** Coleção destinada ao cadastro de consumidores para programas de fidelidade e identificação de compras.
5. **vendas:** Coleção central da aplicação que ilustra o poder do modelo de documentos. A antiga tabela associativa Itens_Venda foi completamente abolida. Na nova arquitetura, o documento da "Venda" passou a conter um array interno chamado itens_consumidos. Cada item comprado agora é um subdocumento gravado diretamente dentro da venda. Isso permite que a aplicação recupere todo o histórico financeiro e os detalhes dos produtos de uma comanda com apenas uma única operação de leitura no disco, maximizando a performance do sistema.

3 RESULTADOS OBTIDOS

Nesta seção, apresentam-se os resultados práticos da migração estrutural. O projeto de banco de dados NoSQL foi implementado no SGBD MongoDB, mantendo a integridade dos dados e o cenário de negócio desenvolvido anteriormente no modelo relacional. A grande alteração arquitetural reside na eliminação de tabelas associativas e na aplicação do padrão de Documentos Aninhados, otimizando a leitura das informações.

Algoritmo 1 – Inserção de Dados (MongoDB)

```
db.funcionarios.drop();
db.mesas.drop();
db.produtos.drop();
db.clientes.drop();
db.vendas.drop();

1. Inserção Funcionários

db.funcionarios.insertMany([
  { _id: 1, nome: "Ana Silva", cargo: "Gerente", data_admissao: new
Date("2024-01-15T00:00:00Z"), ativo: true },
  { _id: 2, nome: "Carlos Souza", cargo: "Barista", data_admissao: new
Date("2024-03-10T00:00:00Z"), ativo: true },
  { _id: 3, nome: "Beatriz Lima", cargo: "Atendente", data_admissao: new
Date("2024-06-20T00:00:00Z"), ativo: true }
]);

2. Inserção Mesas

db.mesas.insertMany([
  { _id: 1, numero_mesa: 1, capacidade: 2, status: "Livre" },
  { _id: 2, numero_mesa: 2, capacidade: 4, status: "Ocupada" },
  { _id: 3, numero_mesa: 3, capacidade: 4, status: "Livre" },
  { _id: 4, numero_mesa: 4, capacidade: 6, status: "Livre" }
]);

3. Inserção Produtos

db.produtos.insertMany([
  { _id: 1, nome_produto: "Café Expresso", descricao: "Café puro
tradicional", categoria: "Bebidas Quentes", preco: 6.50, estoque_atual: 100
},
  { _id: 2, nome_produto: "Cappuccino", descricao: "Café, leite vaporizado
e canela", categoria: "Bebidas Quentes", preco: 12.00, estoque_atual: 50 },
  { _id: 3, nome_produto: "Pão de Queijo", descricao: "Porção com 5
unidades", categoria: "Salgados", preco: 8.00, estoque_atual: 80 },
  { _id: 4, nome_produto: "Bolo de Cenoura", descricao: "Fatia com
cobertura de chocolate", categoria: "Sobremesas", preco: 10.00,
estoque_atual: 20 },
  { _id: 5, nome_produto: "Suco de Laranja", descricao: "Suco natural
400ml", categoria: "Bebidas Geladas", preco: 9.00, estoque_atual: 30 }
]);

4. Inserção Clientes
```



```
db.clientes.insertMany([
  { _id: 1, nome: "João Pedro Alves", cpf: "12345678901", telefone:
    "12999998888", email: "joao.alves@email.com", data_cadastro: new
    Date("2024-05-14T00:00:00Z") },
  { _id: 2, nome: "Mariana Costa", cpf: "98765432100", telefone:
    "12988887777", email: "mariana.costa@email.com", data_cadastro: new
    Date("2024-05-14T00:00:00Z") },
  { _id: 3, nome: "Lucas Pereira", cpf: "11122233344", telefone:
    "12977776666", email: "lucas.p@email.com", data_cadastro: new Date("2024-
    05-14T00:00:00Z") }
]);
```

5. Inserção Vendas com Itens Embutidos (Desnormalização)

```
db.vendas.insertMany([
  {
    _id: 1,
    id_cliente: 1,
    cliente_nome: "João Pedro Alves",
    funcionario: { id_funcionario: 2, nome: "Carlos Souza", cargo:
      "Barista" },
    id_mesa: 2,
    data_hora: new Date("2025-05-10T09:30:00Z"),
    valor_total: 21.00,
    metodo_pagamento: "Pix",
    tipo_servico: "Local",
    itens_consumidos: [
      { id_produto: 1, nome_produto: "Café Expresso", categoria: "Bebidas
        Quentes", quantidade: 1, preco_momento: 6.50 },
      { id_produto: 2, nome_produto: "Cappuccino", categoria: "Bebidas
        Quentes", quantidade: 1, preco_momento: 12.00 },
      { id_produto: 3, nome_produto: "Pão de Queijo", categoria:
        "Salgados", quantidade: 1, preco_momento: 2.50 }
    ]
  },
  {
    _id: 2,
    id_cliente: 2,
    cliente_nome: "Mariana Costa",
    funcionario: { id_funcionario: 3, nome: "Beatriz Lima", cargo:
      "Atendente" },
    id_mesa: null,
    data_hora: new Date("2025-05-10T10:15:00Z"),
    valor_total: 12.00,
    metodo_pagamento: "Cartão de Débito",
    tipo_servico: "Viagem",
    itens_consumidos: [
      { id_produto: 2, nome_produto: "Cappuccino", categoria: "Bebidas
        Quentes", quantidade: 1, preco_momento: 12.00 }
    ]
  },
  {
    _id: 3,
    id_cliente: null,
    cliente_nome: null,
    funcionario: { id_funcionario: 3, nome: "Beatriz Lima", cargo:
      "Atendente" },
    id_mesa: 1,
    data_hora: new Date("2025-05-10T11:00:00Z"),
    valor_total: 32.50,
    metodo_pagamento: "Dinheiro",
    tipo_servico: "Local",
    itens_consumidos: [
      { id_produto: 4, nome_produto: "Bolo de Cenoura", categoria:
        "Sobremesas", quantidade: 2, preco_momento: 10.00 },
      { id_produto: 1, nome_produto: "Café Expresso", categoria: "Bebidas
        Quentes", quantidade: 1, preco_momento: 6.50 },

```

```

    { id_produto: 3, nome_produto: "Pão de Queijo", categoria:
"Salgados", quantidade: 1, preco_momento: 6.00 }
  ]
}

});

```

3.1 Consultas Desenvolvidas

- Consulta 1 - Faturamento Total por Funcionário

Objetivo (Explicação): Agrupa as vendas por funcionário e soma o valor total arrecadado por cada um, ordenando do maior para o menor.

```

db.vendas.aggregate([
  { $group: { _id: { nome: "$funcionario.nome", cargo: "$funcionario.cargo"
}, Faturamento_Total: { $sum: "$valor_total" } } },
  { $sort: { Faturamento_Total: -1 } }
])

```

Algoritmo 2 – Consulta 1 (MongoDB)

```

< {
  _id: {
    nome: 'Beatriz Lima',
    cargo: 'Atendente'
  },
  Faturamento_Total: 44.5
}
{
  _id: {
    nome: 'Carlos Souza',
    cargo: 'Barista'
  },
  Faturamento_Total: 21
}

```

Figura 1 – Consulta 1 (MongoDB)

- Consulta 2 - Produtos mais vendidos em quantidade.

Objetivo (Explicação): Desmembra os itens da venda (\$unwind) e soma a quantidade vendida de cada produto para listar o Top 5 de saídas.

```

1 db.vendas.aggregate([
2
3   { $unwind: "$itens_consumidos" },
4
5   { $group: { _id: { produto: "$itens_consumidos.nome_produto", categoria:
6 "$itens_consumidos.categoria" }, Quantidade_Vendida: { $sum:
7 "$itens_consumidos.quantidade" } } },
8
9   { $sort: { Quantidade_Vendida: -1 } },
10
11   { $limit: 5 }
12 ])
```

Algoritmo 3 – Consulta 2 (MongoDB)

```

< {
  _id: {
    produto: 'Bolo de Cenoura',
    categoria: 'Sobremesas'
  },
  Quantidade_Vendida: 2
}
{
  _id: {
    produto: 'Pão de Queijo',
    categoria: 'Salgados'
  },
  Quantidade_Vendida: 2
}
{
  _id: {
    produto: 'Café Expresso',
    categoria: 'Bebidas Quentes'
  },
  Quantidade_Vendida: 2
}
```

Figura 2 – Consulta 2 (MongoDB)

- Consulta 3 - Clientes que nunca compraram

Objetivo (Explicação): Cruza as coleções de clientes e vendas (\$lookup) e filtra apenas aqueles que possuem zero compras registradas.

```
1 db.clientes.aggregate([
2
3   { $lookup: { from: "vendas", localField: "_id", foreignField: "id_cliente",
4 as: "compras" } },
5
6   { $match: { compras: { $size: 0 } } },
7
8   { $project: { nome: 1, email: 1, data_cadastro: 1 } }
9 ])
```

Algoritmo 4 – Consulta 3 (MongoDB)



```
< {
  _id: 3,
  nome: 'Lucas Pereira',
  email: 'lucas.p@email.com',
  data_cadastro: 2024-05-14T00:00:00.000Z
}
```

Figura 3 – Consulta 3 (MongoDB)

- Consulta 4 - Vendas acima do ticket médio.

Objetivo (Explicação): Filtra as vendas cujo valor total seja estritamente maior que a média de gastos (21.83) do estabelecimento.

```
1 db.vendas.find(
2
3   { valor_total: { $gt: 21.83 } },
4
5   { _id: 1, data_hora: 1, valor_total: 1, metodo_pagamento: 1 }
6
7 )
8
```

Algoritmo 5 – Consulta 4 (MongoDB)

```
< {
  _id: 3,
  data_hora: 2025-05-10T11:00:00.000Z,
  valor_total: 32.5,
  metodo_pagamento: 'Dinheiro'
}
```

Figura 4 – Consulta 4 (MongoDB)

- Consulta 5 - Faturamento por modalidade de serviço.
Objetivo (Explicação): Agrupa os pedidos por tipo (Local/Viagem), somando o total arrecadado e filtrando os que superam 20 reais.

```
1 db.vendas.aggregate([
2
3   { $group: { _id: "$tipo_servico", Total_Pedidos: { $sum: 1 }, Faturamento:
4 { $sum: "$valor_total" } } },
5
6   { $match: { Faturamento: { $gt: 20 } } }
7
8 ])
```

Algoritmo 6 – Consulta 5 (MongoDB)

```
< {
  _id: 'Local',
  Total_Pedidos: 2,
  Faturamento: 53.5
}
```

Figura 5 – Consulta 5 (MongoDB)

- Consulta 6 - Produtos com baixo nível de estoque.
Objetivo (Explicação): Lista os produtos que possuem menos de 50 unidades em estoque para gerar um alerta de reposição.

```

1 db.produtos.find(
2   { estoque_atual: { $lt: 50 } },
3   { _id: 0, nome_produto: 1, categoria: 1, estoque_atual: 1 }
4   ).sort({ estoque_atual: 1 })
5
6
7
8

```

Algoritmo 7 – Consulta 6 (MongoDB)

```

< {
  nome_produto: 'Bolo de Cenoura',
  categoria: 'Sobremesas',
  estoque_atual: 20
}
{
  nome_produto: 'Suco de Laranja',
  categoria: 'Bebidas Geladas',
  estoque_atual: 30
}

```

Figura 6 – Consulta 6 (MongoDB)

- Consulta 7 - Histórico de vendas por cliente cadastrado.
Objetivo (Explicação): Filtra apenas as comandas que possuem um cliente oficialmente identificado, ignorando vendas anônimas.

```

1 db.vendas.find(
2   { id_cliente: { $ne: null } },
3   { _id: 0, cliente_nome: 1, data_hora: 1, valor_total: 1, metodo_pagamento:
4   1 }
5   ).sort({ data_hora: -1 })
6
7

```

Algoritmo 8 – Consulta 7 (MongoDB)

```
< {
  cliente_nome: 'Mariana Costa',
  data_hora: 2025-05-10T10:15:00.000Z,
  valor_total: 12,
  metodo_pagamento: 'Cartão de Débito'
}
{
  cliente_nome: 'João Pedro Alves',
  data_hora: 2025-05-10T09:30:00.000Z,
  valor_total: 21,
  metodo_pagamento: 'Pix'
}
```

Figura 7 – Consulta 7 (MongoDB)

- Consulta 8 - Frequência de uso dos métodos de pagamento

Objetivo (Explicação): Agrupa as transações pelo método de pagamento para contabilizar quantas vezes cada opção foi utilizada.

```
1 db.vendas.aggregate([
2
3   { $group: { _id: "$metodo_pagamento", Frequencia_Uso: { $sum: 1 } } },
4
5   { $sort: { Frequencia_Uso: -1 } }
6
7 ])
8
```

Algoritmo 9 – Consulta 8 (MongoDB)

```
< {
  _id: 'Cartão de Débito',
  Frequencia_Uso: 1
}
{
  _id: 'Pix',
  Frequencia_Uso: 1
}
{
  _id: 'Dinheiro',
  Frequencia_Uso: 1
}
```

Figura 8 – Consulta 8 (MongoDB)

- Consulta 9 - Detalhamento completo da comanda 1

Objetivo (Explicação): Filtra a venda específica, desmembra seus itens e calcula o subtotal de cada produto multiplicado pela sua quantidade.

```
1 db.vendas.aggregate([
2
3   { $match: { _id: 1 } },
4
5   { $unwind: "$itens_consumidos" },
6
7   { $project: { _id: 0, Comanda: "$_id", Produto:
8 "$itens_consumidos.nome_produto", Qtd: "$itens_consumidos.quantidade",
9 Preco_Unitario: "$itens_consumidos.preco_momento", Subtotal: { $multiply:
10 ["$itens_consumidos.quantidade", "$itens_consumidos.preco_momento"] } } }
11 ])
```

Algoritmo 10 – Consulta 9 (MongoDB)


```
< {
  Comanda: 1,
  Produto: 'Café Expresso',
  Qtd: 1,
  Preço_Unitario: 6.5,
  Subtotal: 6.5
}
{
  Comanda: 1,
  Produto: 'Cappuccino',
  Qtd: 1,
  Preço_Unitario: 12,
  Subtotal: 12
}
{
  Comanda: 1,
  Produto: 'Pão de Queijo',
  Qtd: 1,
  Preço_Unitario: 2.5,
  Subtotal: 2.5
}
```

Figura 9 – Consulta 9 (MongoDB)

- Consulta 10 - Receita gerada por categoria de produto.
Objetivo (Explicação): Desmembra os itens, calcula o valor financeiro ganho por item e agrupa o faturamento por categoria (ex: Sobremesas, Bebidas).

```
db.vendas.aggregate([
1   { $unwind: "$itens_consumidos" },
2   { $group: { _id: "$itens_consumidos.categoria", Faturamento_Total: { $sum:
3 { $multiply: ["$itens_consumidos.quantidade",
4 "$itens_consumidos.preco_momento"] } } } },
5
6   { $sort: { Faturamento_Total: -1 } }
])
```

Algoritmo 11 – Consulta 10 (MongoDB)

```
< {  
  _id: 'Bebidas Quentes',  
  Faturamento_Total: 37  
}  
{  
  _id: 'Sobremesas',  
  Faturamento_Total: 20  
}  
{  
  _id: 'Salgados',  
  Faturamento_Total: 8.5  
}
```

Figura 10 – Consulta 10 (MongoDB)

- Consulta 11 - Ocupação histórica das mesas
Objetivo (Explicação): Filtra as vendas que utilizaram mesas físicas (não nulas) e conta quantas vezes cada mesa específica foi ocupada.

```
1  
2  
3  
4 db.vendas.aggregate([  
5   { $match: { id_mesa: { $ne: null } } },  
6  
7   { $group: { _id: "$id_mesa", Total_Usos: { $sum: 1 } } },  
8  
9   { $sort: { Total_Usos: -1 } }  
10 ])  
11  
12
```

Algoritmo 12 – Consulta 11 (MongoDB)

```
< {
  _id: 1,
  Total_Usos: 1
}
{
  _id: 2,
  Total_Usos: 1
}
```

Figura 11 – Consulta 11 (MongoDB)

- Consulta 12 - Funcionários admitidos no ano de 2024.

Objetivo (Explicação): Filtra a coleção de funcionários para exibir apenas as contratações ocorridas dentro do ano de 2024.

```
1
2 db.funcionarios.aggregate([
3
4   { $match: { data_admissao: { $gte: new Date("2024-01-01T00:00:00Z"), $lt:
5 new Date("2025-01-01T00:00:00Z") } } },
6
7   { $project: { _id: 0, Colaborador: "$nome", Funcao: "$cargo",
8 data_admissao: 1 } }
9
10 ])
```

Algoritmo 13 – Consulta 12 (MongoDB)

```
< {
  data_admissao: 2024-01-15T00:00:00.000Z,
  Colaborador: 'Ana Silva',
  Funcao: 'Gerente'
}
{
  data_admissao: 2024-03-10T00:00:00.000Z,
  Colaborador: 'Carlos Souza',
  Funcao: 'Barista'
}
{
  data_admissao: 2024-06-20T00:00:00.000Z,
  Colaborador: 'Beatriz Lima',
  Funcao: 'Atendente'
}
```

Figura 12 – Consulta 12 (MongoDB)

- Consulta 13 - Clientes com nomes iniciados por M ou L
Objetivo (Explicação): Utiliza Expressão Regular (\$regex) para buscar dinamicamente nomes que comecem especificamente com as letras M ou L.

```
1
2
3 db.clientes.find(
4   { nome: { $regex: "^[ML]", $options: "i" } },
5   { _id: 0, nome: 1, email: 1, telefone: 1 }
6
7
8 ).sort({ nome: 1 })
9
```

Algoritmo 14 – Consulta 13 (MongoDB)

```
< {
  nome: 'Lucas Pereira',
  telefone: '12977776666',
  email: 'lucas.p@email.com'
}
{
  nome: 'Mariana Costa',
  telefone: '12988887777',
  email: 'mariana.costa@email.com'
}
```

Figura 13 – Consulta 13 (MongoDB)

- Consulta 14 - Ticket médio de venda por funcionário.

Objetivo (Explicação): Calcula a média financeira (\$avg) gerada por cada atendente e arredonda o resultado final para duas casas decimais.

```
1 db.vendas.aggregate([
2   { $group: { _id: "$funcionario.nome", Ticket_Medio: { $avg: "$valor_total"
3 } } },
4
5   { $project: { Ticket_Medio: { $round: ["$Ticket_Medio", 2] } } },
6
7   { $sort: { Ticket_Medio: -1 } }
8 ])
```

Algoritmo 15 – Consulta 14 (MongoDB)

```
< {
  _id: 'Beatriz Lima',
  Ticket_Medio: 22.25
}
{
  _id: 'Carlos Souza',
  Ticket_Medio: 21
}
```

Figura 14 – Consulta 14 (MongoDB)

- Consulta 15 - Produtos sem nenhuma saída.

Objetivo (Explicação): Cruza o catálogo de produtos com os itens vendidos (\$lookup) para identificar mercadorias encalhadas (zero vendas).

```

1 db.produtos.aggregate([
2
3   { $lookup: { from: "vendas", localField: "_id", foreignField:
4 "itens_consumidos.id_produto", as: "vendas" } },
5
6   { $match: { vendas: { $size: 0 } } },
7
8   { $project: { _id: 1, nome_produto: 1, categoria: 1 } }
9 ])

```

Algoritmo 16 – Consulta 15 (MongoDB)

```

< {
  _id: 5,
  nome_produto: 'Suco de Laranja',
  categoria: 'Bebidas Geladas'
}

```

Figura 15 – Consulta 15 (MongoDB)

- Consulta 16 - Filtrar data específica (10/05/2025)

Objetivo (Explicação): Filtra os documentos de vendas que ocorreram estritamente no intervalo do dia 10/05/2025.

```

1 db.vendas.find(
2
3   { data_hora: { $gte: new Date("2025-05-10T00:00:00Z"), $lt: new Date("2025-
4 05-11T00:00:00Z") } },
5
6   { _id: 1, valor_total: 1, metodo_pagamento: 1 }
7 )

```

Algoritmo 17 – Consulta 16 (MongoDB)

```
< {
  _id: 1,
  valor_total: 21,
  metodo_pagamento: 'Pix'
}
{
  _id: 2,
  valor_total: 12,
  metodo_pagamento: 'Cartão de Débito'
}
{
  _id: 3,
  valor_total: 32.5,
  metodo_pagamento: 'Dinheiro'
}
```

Figura 16 – Consulta 16 (MongoDB)

- Consulta 17 - Controle de preço mínimo (≥ 10.00)

Objetivo (Explicação): Lista os itens do catálogo de produtos que custam 10 reais ou mais (\$gte).

```
1 db.produtos.find(
2
3 { preco: { $gte: 10.00 } },
4
5 { _id: 0, nome_produto: 1, categoria: 1, preco: 1 }
6
7 )
8
9
```

Algoritmo 18 – Consulta 17 (MongoDB)

```
< {
  nome_produto: 'Cappuccino',
  categoria: 'Bebidas Quentes',
  preco: 12
}
{
  nome_produto: 'Bolo de Cenoura',
  categoria: 'Sobremesas',
  preco: 10
}
```

Figura 17 – Consulta 17 (MongoDB)

- Consulta 18 - Simulação de Procedure de Busca por Nome.

Objetivo (Explicação): Simula uma busca rápida retornando contatos de clientes que contenham o termo "Mariana" em qualquer parte do nome.

```
1
2
3
4
5 db.clientes.find(
6   { nome: { $regex: "Mariana", $options: "i" } },
7
8   { _id: 0, nome: 1, telefone: 1, email: 1 }
9
10 )
11
12
```

Algoritmo 19 – Consulta 18 (MongoDB)

```
< {
  nome: 'Mariana Costa',
  telefone: '12988887777',
  email: 'mariana.costa@email.com'
}
```

Figura 18 – Consulta 18 (MongoDB)

- Consulta 19 - Simulação de Procedure de Vendas por Pagamento

Objetivo (Explicação): Filtra compras pagas com cédulas e projeta a data da venda formatada no padrão brasileiro (\$dateToString).

```

1 db.vendas.aggregate([
2
3   { $match: { metodo_pagamento: "Dinheiro" } },
4
5   { $project: { _id: 1, valor_total: 1, data_hora_formatada: { $dateToString:
6     { format: "%d/%m/%Y %H:%M", date: "$data_hora" } } } }
7 ])
```

Algoritmo 20 – Consulta 19 (MongoDB)

```

< {
  _id: 3,
  valor_total: 32.5,
  data_hora_formatada: '10/05/2025 11:00'
}
```

Figura 19 – Consulta 19 (MongoDB)

- Consulta 20 - Simulação de Trigger - Baixa de Estoque.

Objetivo (Explicação): Decrementa (\$inc) pontualmente o estoque de um produto específico, simulando a baixa atômica de uma venda.

```

1 // Atualizando manualmente o estoque do Pão de Queijo (id_produto: 3)
2
3 db.produtos.updateOne(
4
5   { _id: 3 },
6
7   { $inc: { estoque_atual: -5 } }
8 )
```

Algoritmo 21 – Consulta 20 (MongoDB)

```
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figura 20 – Consulta 20 (MongoDB)

- Consulta 21 - Simulação de Trigger de Segurança - Desativar ao invés de Deletar.

Objetivo (Explicação): Atualiza (\$set) o status de um funcionário para inativo em vez de apagar definitivamente o seu registro do banco.

```
1 db.funcionarios.updateOne(
2
3   { _id: 1 },
4
5   { $set: { ativo: false } }
6
7
8 )
```

Algoritmo 22 – Consulta 21 (MongoDB)

```
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figura 21 – Consulta 21 (MongoDB)

- Consulta 22 - Inspeção da estrutura da tabela
Objetivo (Explicação): Busca o primeiro produto do banco apenas para analisar visualmente os campos estruturais que o compõem.

```
1 db.produtos.find({ _id: 1 })
```

Algoritmo 23 – Consulta 22 (MongoDB)

```
< {  
  _id: 1,  
  nome_produto: 'Café Expresso',  
  descricao: 'Café puro tradicional',  
  categoria: 'Bebidas Quentes',  
  preco: 6.5,  
  estoque_atual: 100  
}
```

Figura 22 – Consulta 22 (MongoDB)

- Consulta 23 - Filtragem de preços por intervalo (BETWEEN)
Objetivo (Explicação): Lista produtos cujos preços estejam na faixa entre 8 e 12 reais, ordenando do mais caro para o mais barato.

```
1  
2  
3 db.produtos.find(  
4   { preco: { $gte: 8.00, $lte: 12.00 } },  
5  
6   { _id: 0, nome_produto: 1, categoria: 1, preco: 1 }  
7  
8 ).sort({ preco: -1 })  
9
```

Algoritmo 24 – Consulta 23 (MongoDB)

```

< {
  nome_produto: 'Cappuccino',
  categoria: 'Bebidas Quentes',
  preco: 12
}
{
  nome_produto: 'Bolo de Cenoura',
  categoria: 'Sobremesas',
  preco: 10
}
{
  nome_produto: 'Suco de Laranja',
  categoria: 'Bebidas Geladas',
  preco: 9
}
{
  nome_produto: 'Pão de Queijo',
  categoria: 'Salgados',
  preco: 8
}

```

Figura 23 – Consulta 23 (MongoDB)

- Consulta 24 - Verificação de ausência de dados (IS NULL).
Objetivo (Explicação): Filtra vendas onde a mesa é nula, indicando pedidos para viagem, e formata a exibição da data.

```

1 db.vendas.aggregate([
2
3   { $match: { id_mesa: null } },
4
5   { $project: { _id: 0, Comanda: "$_id", Horario: { $dateToString: { format:
6     "%d/%m/%Y %H:%M", date: "$data_hora" } }, Servico: "$tipo_servico" } }
7
8 ])
9

```

Algoritmo 25 – Consulta 24 (MongoDB)

```
{
  Comanda: 2,
  Horario: '10/05/2025 10:15',
  Servico: 'Viagem'
}
```

Figura 24 – Consulta 24 (MongoDB)

- Consulta 25 - Verificação de presença de dados (IS NOT NULL)
Objetivo (Explicação): Seleciona apenas os consumidores que forneceram um e-mail válido (\$ne: null) no momento do cadastro.

```
1 db.clientes.find(
2
3 { email: { $ne: null } },
4
5 { _id: 0, Cliente: "$nome", Contato: "$telefone", Correio_Eletronico:
6 "$email" }
7
8 )
9
```

Algoritmo 26 – Consulta 25 (MongoDB)

```
< {
  Cliente: 'João Pedro Alves',
  Contato: '12999998888',
  Correio_Eletronico: 'joao.alves@email.com'
}
{
  Cliente: 'Mariana Costa',
  Contato: '12988887777',
  Correio_Eletronico: 'mariana.costa@email.com'
}
{
  Cliente: 'Lucas Pereira',
  Contato: '12977776666',
  Correio_Eletronico: 'lucas.p@email.com'
}
```

Figura 25 – Consulta 25 (MongoDB)

- Consulta 26 - Busca por padrão de texto específico (%Café%)

Objetivo (Explicação): Busca por produtos que contenham a palavra "Café" em qualquer parte do nome, ignorando letras maiúsculas ou minúsculas.

```
1 db.produtos.find(  
2 { nome_produto: { $regex: "Café", $options: "i" } },  
3  
4 { _id: 1, nome_produto: 1, categoria: 1 }  
5  
6 )  
7  
8 )
```

Algoritmo 27 – Consulta 26 (MongoDB)



```
< {  
  _id: 1,  
  nome_produto: 'Café Expresso',  
  categoria: 'Bebidas Quentes'  
}
```

Figura 26 – Consulta 26 (MongoDB)

- Consulta 27 - Cálculo de intervalo entre datas (DATEDIFF).

Objetivo (Explicação): Usa o operador cronológico \$dateDiff para calcular com exatidão há quantos dias o cliente está cadastrado no sistema.

```
1 db.clientes.aggregate([  
2 { $project: { _id: 0, Cliente: "$nome", data_cadastro: 1,  
3 Dias_Como_Cliente: { $dateDiff: { startDate: "$data_cadastro", endDate: new  
4 Date(), unit: "day" } } } }  
5 ])
```

Algoritmo 28 – Consulta 27 (MongoDB)

```
< {
  data_cadastro: 2024-05-14T00:00:00.000Z,
  Cliente: 'João Pedro Alves',
  Dias_Como_Cliente: Long('770')
}
{
  data_cadastro: 2024-05-14T00:00:00.000Z,
  Cliente: 'Mariana Costa',
  Dias_Como_Cliente: Long('770')
}
{
  data_cadastro: 2024-05-14T00:00:00.000Z,
  Cliente: 'Lucas Pereira',
  Dias_Como_Cliente: Long('770')
}
```

Figura 27 – Consulta 27 (MongoDB)

- Consulta 28 - Classificação dinâmica de preços (CASE WHEN)
Objetivo (Explicação): Aplica o \$switch para criar uma nova coluna visual classificando os produtos em "Econômico", "Padrão" ou "Premium".

```
1 db.produtos.aggregate([
2
3   { $project: { _id: 0, Produto: "$nome_produto", Valor: "$preco",
4   Faixa_Preco: { $switch: { branches: [ { case: { $lt: ["$preco", 10.00] },
5   then: "Econômico" }, { case: { $lte: ["$preco", 15.00] }, then: "Padrão" } ],
6   default: "Premium" } } } }
7 ]])
```

Algoritmo 29 – Consulta 28 (MongoDB)

```
< {  
  Produto: 'Café Expresso',  
  Valor: 6.5,  
  Faixa_Preco: 'Econômico'  
}  
{  
  Produto: 'Cappuccino',  
  Valor: 12,  
  Faixa_Preco: 'Padrão'  
}  
{  
  Produto: 'Pão de Queijo',  
  Valor: 8,  
  Faixa_Preco: 'Econômico'  
}  
{  
  Produto: 'Bolo de Cenoura',  
  Valor: 10,  
  Faixa_Preco: 'Padrão'  
}  
{  
  Produto: 'Suco de Laranja',  
  Valor: 9,  
  Faixa_Preco: 'Econômico'  
}
```

Figura 28 – Consulta 28 (MongoDB)

- Consulta 29 - Tradução de status de operação (CASE WHEN)
Objetivo (Explicação): Utiliza o \$switch para trocar os termos "Livre" e "Ocupada" do banco por mensagens mais amigáveis ao usuário final.


```

1 db.mesas.aggregate([
2
3   { $project: { _id: 0, Mesa: "$numero_mesa", Lugares: "$capacidade",
4 Status_Formatado: { $switch: { branches: [ { case: { $eq: ["$status",
5 "Livre"] }, then: "Disponível para Clientes" }, { case: { $eq: ["$status",
6 "Ocupada"] }, then: "Em Atendimento" } ], default: "Manutenção" } } } }
7
8 ])
```

Algoritmo 30 – Consulta 29 (MongoDB)

```

< {
  Mesa: 1,
  Lugares: 2,
  Status_Formatado: 'Disponível para Clientes'
}
{
  Mesa: 2,
  Lugares: 4,
  Status_Formatado: 'Em Atendimento'
}
{
  Mesa: 3,
  Lugares: 4,
  Status_Formatado: 'Disponível para Clientes'
}
{
  Mesa: 4,
  Lugares: 6,
  Status_Formatado: 'Disponível para Clientes'
}
```

Figura 29 – Consulta 29 (MongoDB)

- Consulta 30 - Desempenho semanal de vendas por produto (DATEPART WEEK)
Objetivo (Explicação): Extrai a semana do ano (\$isoWeek) da data de venda e agrupa para mostrar o volume de produtos comercializados semanalmente.

```

1 db.vendas.aggregate([
2
3   { $unwind: "$itens_consumidos" },
4
5   { $group: { _id: { Semana_do_Ano: { $isoWeek: "$data_hora" }, Produto:
6 "$itens_consumidos.nome_produto" }, Quantidade_Vendida: { $sum:
7 "$itens_consumidos.quantidade" } } },
8
9   { $sort: { Quantidade_Vendida: -1 } }
10 ])
```

Algoritmo 31 – Consulta 30 (MongoDB)

```

< {
  _id: {
    Semana_do_Ano: 19,
    Produto: 'Bolo de Cenoura'
  },
  Quantidade_Vendida: 2
}
{
  _id: {
    Semana_do_Ano: 19,
    Produto: 'Pão de Queijo'
  },
  Quantidade_Vendida: 2
}
{
  _id: {
    Semana_do_Ano: 19,
    Produto: 'Café Expresso'
  },
  Quantidade_Vendida: 2
}
```

Figura 30 – Consulta 30 (MongoDB)

4 CONCLUSÃO

O presente projeto demonstrou com sucesso a viabilidade técnica e os benefícios arquiteturais da migração do sistema de gerenciamento da cafeteria "Café Aconchego" de um modelo estritamente relacional para um paradigma não relacional (NoSQL) orientado a documentos. Os resultados obtidos através da implementação no SGBD MongoDB comprovaram que a adoção da técnica de desnormalização e o uso de documentos aninhados (Embedded Documents) trouxeram ganhos expressivos de flexibilidade de esquema e desempenho de leitura. A eliminação da antiga tabela associativa de itens, substituída por um vetor interno no documento da venda, extinguiu a necessidade de múltiplas junções (JOINS) custosas.

Dessa forma, tornou-se possível recuperar e manipular comandas completas através de operações atômicas simples, evidenciando a forte adequação do ecossistema NoSQL para cenários dinâmicos e escaláveis exigidos pelo setor comercial moderno. Apesar dos desafios inerentes à redundância de dados imposta por esse modelo, a transição estrutural atendeu plenamente aos objetivos de modernização estipulados

Por fim, visando o aprimoramento contínuo e a evolução tecnológica desta arquitetura, sugerem-se implementações práticas para trabalhos futuros.

Recomenda-se a criação de Índices de Texto (*Text Indexes*) nativos no MongoDB para otimizar pesquisas complexas no catálogo de clientes e produtos; a adoção de regras de Validação de Esquema (*Schema Validation*) para garantir a integridade dos tipos de dados inseridos; e, principalmente, o desenvolvimento de uma API RESTful para atuar como *Back-end*. Essa camada intermediária permitiria a integração do banco de dados com uma futura interface gráfica (Front-end) amigável, abstraindo as operações diretas de linha de comando (*Mongosh*) e automatizando lógicas de negócio cruciais, como as atualizações de estoque.

REFERÊNCIAS

DATE, C. J. **Introdução a Sistemas de Bancos de Dados**. 8. ed. Rio de Janeiro: Campus/Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de Banco de Dados**. 6. ed. São Paulo: Pearson Addison-Wesley, 2011.

FOWLER, Martin; SADALAGE, Pramod J. **NoSQL Essencial: Um Guia Rápido para o Mundo Poliglota**. 1. ed. São Paulo: Novatec Editora, 2013.

MONGODB INC. **MongoDB Documentation**. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 10 jun. 2026.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. **Sistema de Banco de Dados**. 6. ed. Rio de Janeiro: Elsevier, 2012.

SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.